

WED MAY 6 • 11:30AM

DUE 2183

LLMGuard

AN INTEGRITY ATTESTATION FRAMEWORK
FOR LARGE LANGUAGE MODELS

AAYUSH RAI

DR. EUGENE VASSERMAN (ADVISOR)

AGENDA

MOTIVATION

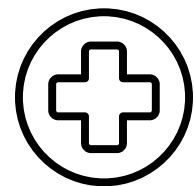
THREAT MODEL

DESIGN DECISION

ARCHITECTURE

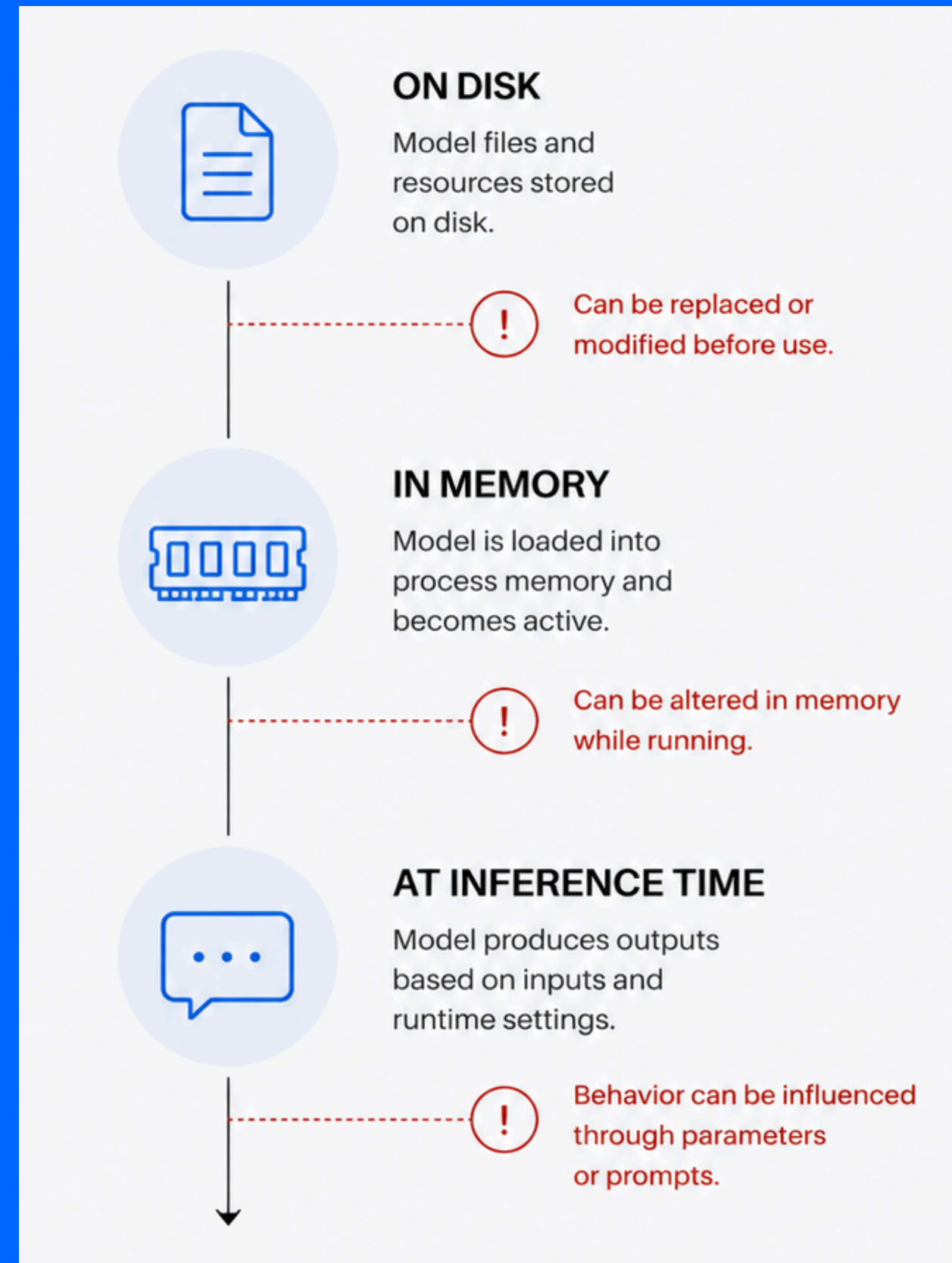
LIMITATIONS

MOTIVATION



Large Language Models in Real World

***Can you trust your model is
running what you deployed?***



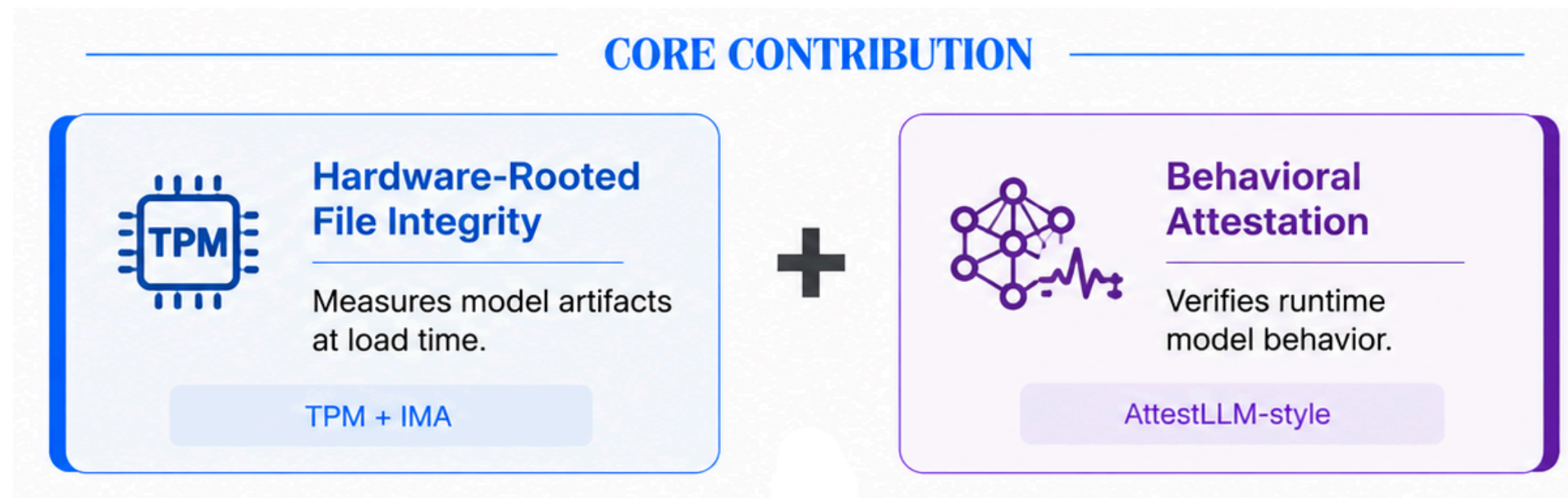
WHERE TAMPERING CAN HAPPEN

A deployed LLM can be modified at multiple points in its life cycle.

PROJECT SCOPE

LLMGuard is a *detection* framework.

LLMGuard targets locally deployed models on commodity Linux hardware.



ATTESTATION PROTOCOL

Attestation is the process of producing tamper-proof evidence that a system is in a known good state.



Prover: the entity whose integrity is being checked.



Verifier: asks for evidence, evaluates it, and decides whether to trust the prover.

TRUSTED FOUNDATION



Linux Kernel

Establishes and enforces measurement boundaries.



TPM (Trusted Platform Module)

Stores integrity measurements in tamper-proof registers.

THE PROVER IS THE ADVERSARY



CONTROLS THE SYSTEM END-TO-END

The attacker owns the process, the model, the runtime, and the operating system.



CAN MODIFY THE MODEL AND IT'S BEHAVIOR

Weights, memory, parameters, tokenizers, anything within their control



CAN FORGE OR HIDE EVIDENCE

They can fake measurements, bypass checks, or manipulate attestation outputs.

ADVERSARY LIMITS

The attacker is powerful, but bounded. These bounds make the design feasible.



ATTACKER CANNOT



Escalate to root

No privilege escalation.



Read or write verifier assets

Cannot access baseline measurements, signing key, audit log, or verifier daemon memory.



Modify system components

Cannot modify the kernel, IMA module, or the TPM.



Tamper with TPM measurements

Integrity measurements can only be extended, never overwritten. They reset only on a hardware power cycle.



OUT OF SCOPE



Physical attacks

- Cold-boot attacks
- Hardware tampering
- Bus probing

Attacker is a software adversary only.



ENFORCED BY



Linux Kernel

Protects system, processes, and verifier assets.



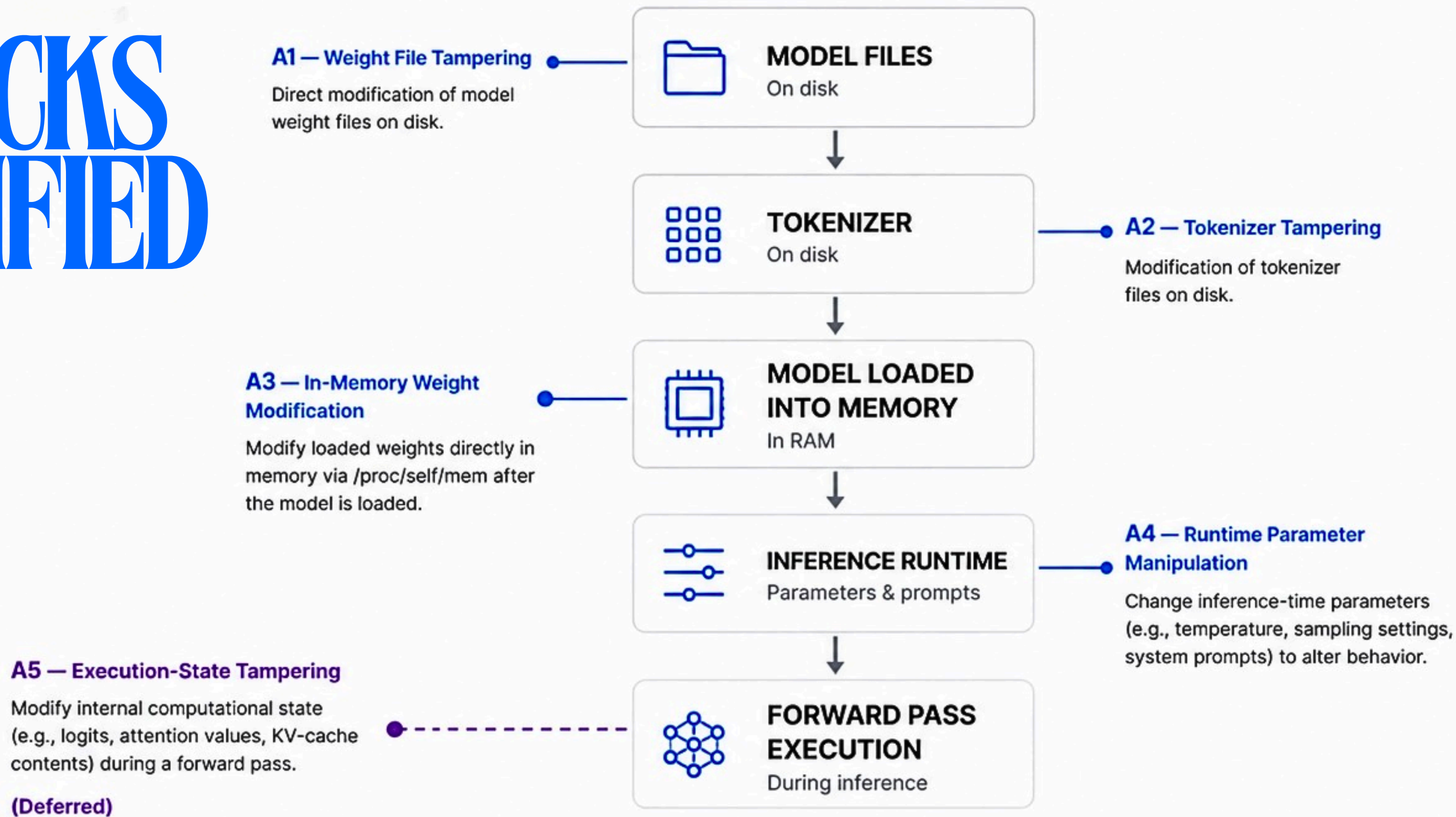
TPM Hardware

Protects measurements and enforces immutability.

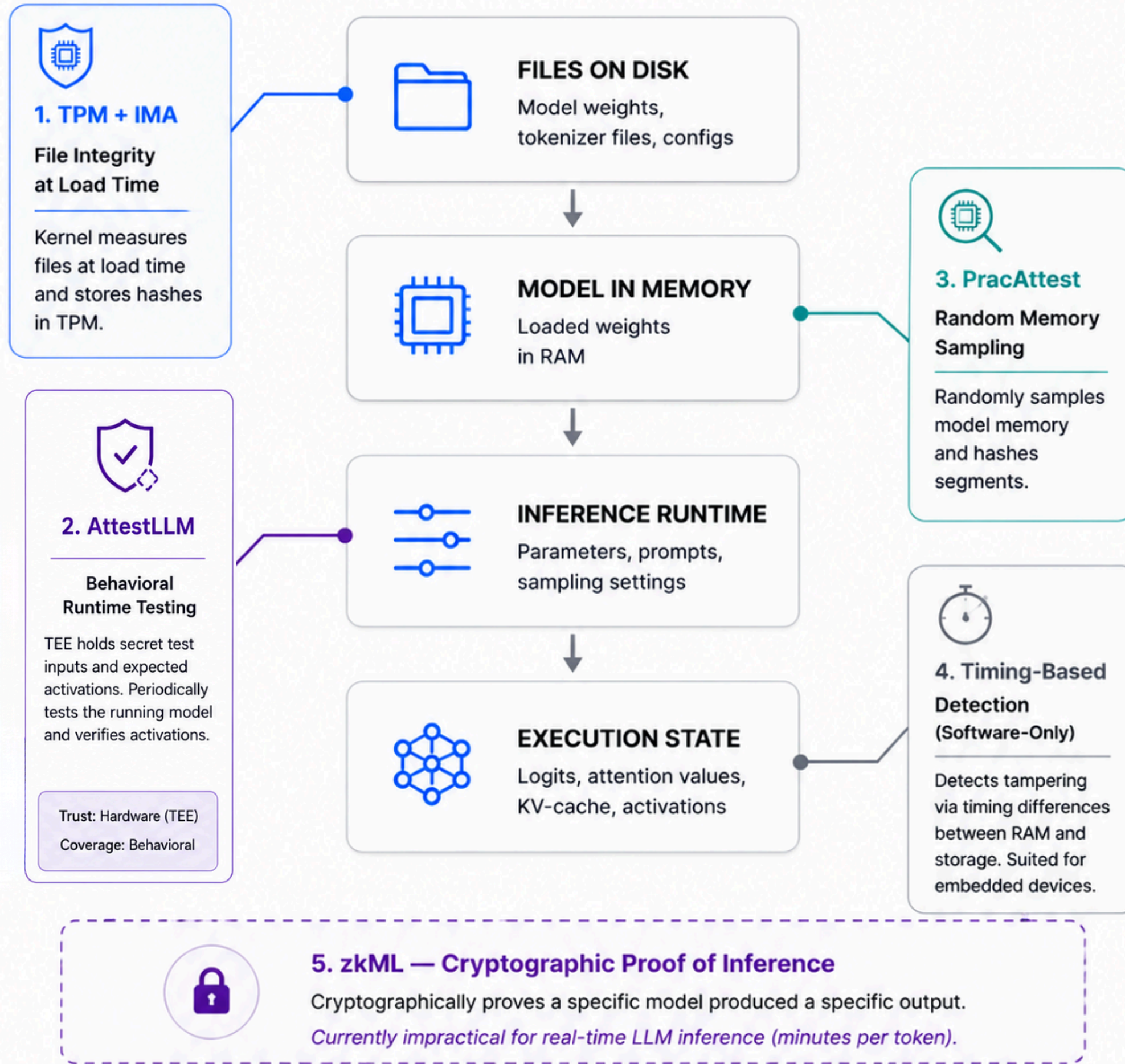


If the kernel or TPM is compromised (e.g., kernel exploit, TPM vulnerability), **none of the guarantees hold.**

ATTACKS IDENTIFIED



FIVE APPROACHES CONSIDERED



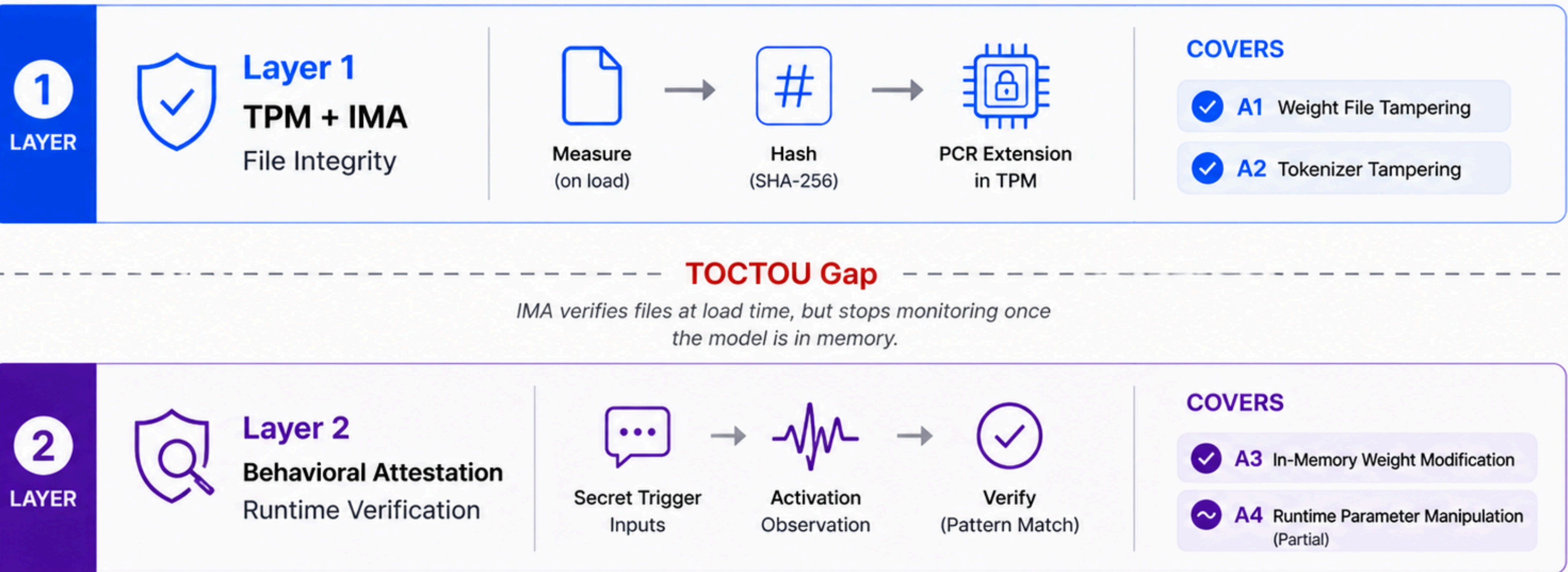
COMPARISON

APPROACH	ATTACK CATEGORIES (FROM THREAT MODEL)				
	A1  Weight File Tampering	A2  Tokenizer Tampering	A3  In-Memory Weight Modification	A4  Runtime Parameter Manipulation	A5  Execution-State Tampering (Deferred)
 1. TPM + IMA File Integrity at Load Time					
 2. AttestLLM Behavioral Runtime Testing					
 3. PracAttest Random Memory Sampling					
 4. Timing-Based (Software-Only) Timing-Based Detection					
 5. zkML Cryptographic Proof of Inference					

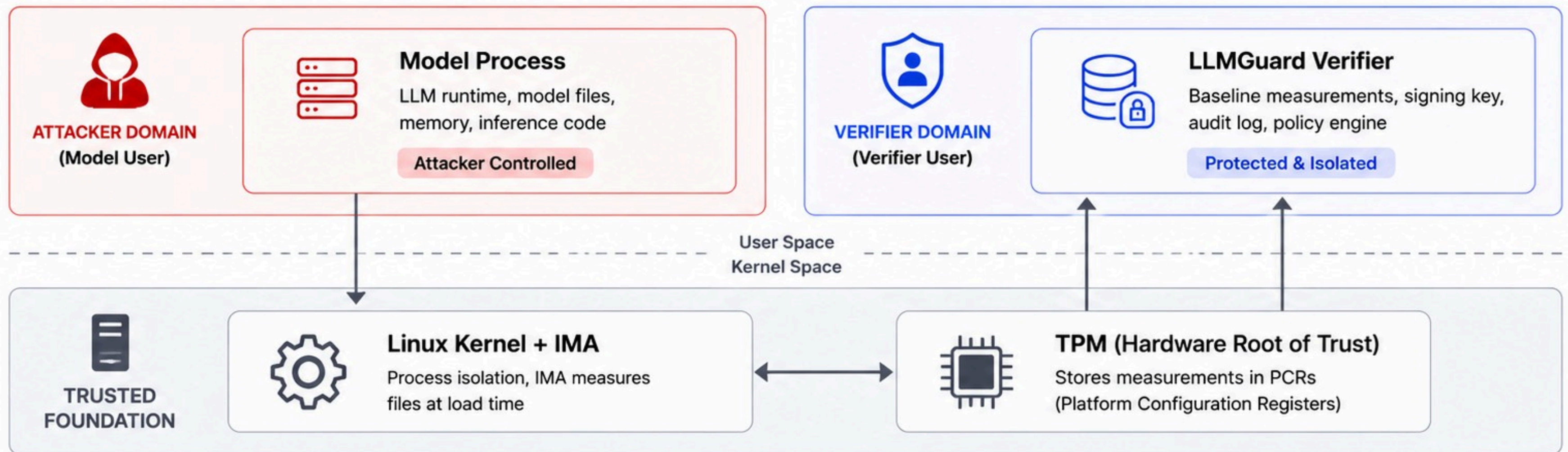
LEGEND

-  **Strong coverage**
Practical and reliable detection
-  **Partial coverage**
Limited or indirect detection
-  **No meaningful coverage**
-  **Theoretical only**
Currently impractical at LLM scale

LAYERED DESIGN



ARCHITECTURE OVERVIEW

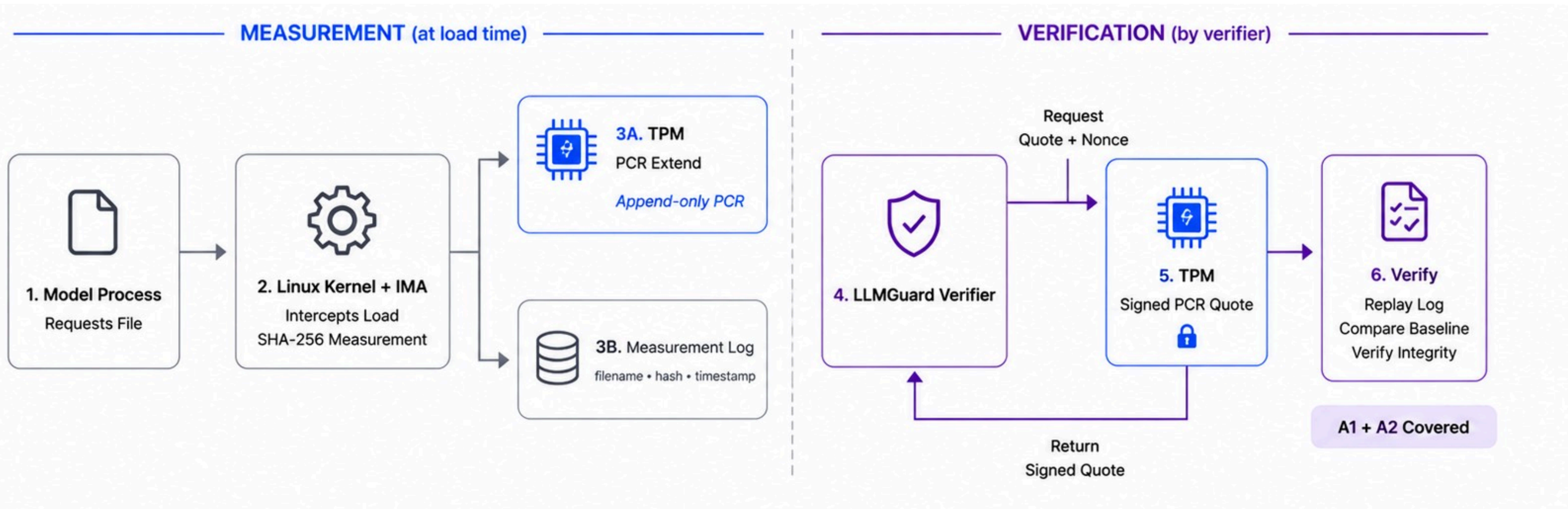


- 1 Load-time Measurements**
Model files are loaded → IMA measures
→ Hash extended into PCRs in TPM

- 2 PCR Quote**
Verifier requests a quote
of PCR values from TPM

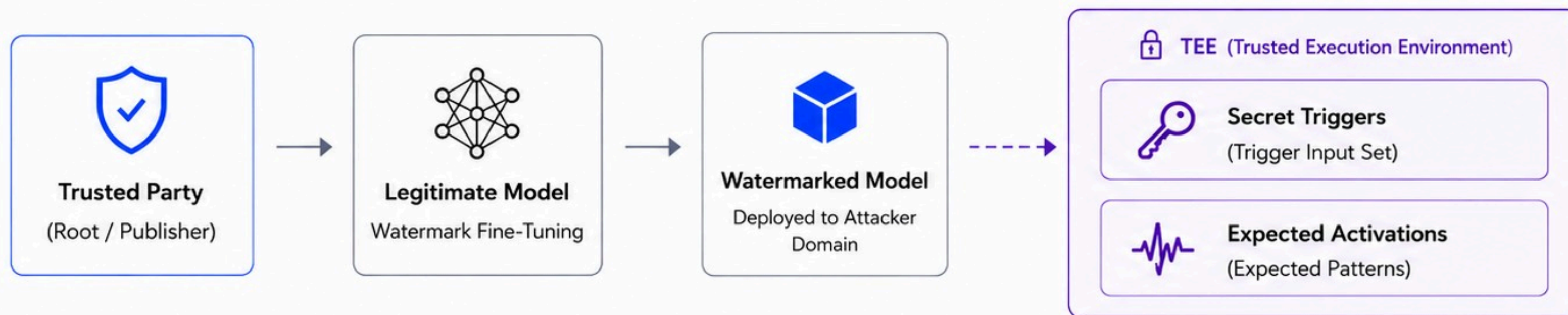
- 3 Attestation Evidence**
Verifier receives PCR quote and
behavioral check results from
the model process

LAYER ONE – FILE INTEGRITY

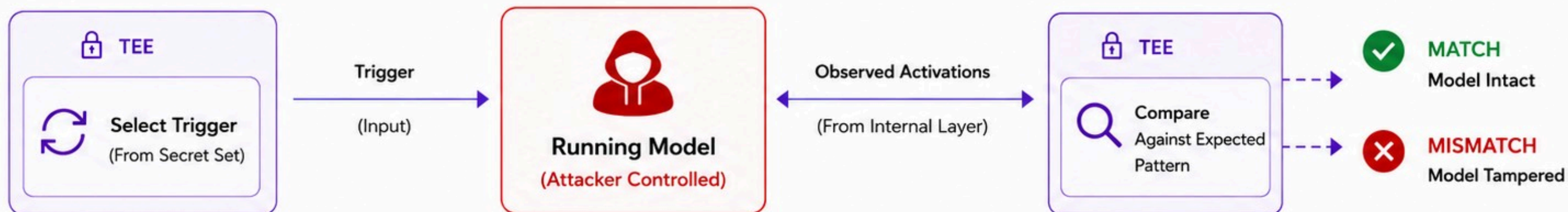


LAYER TWO – BEHAVIORAL ATTESTATION

1. SETUP (One-Time, Trusted)



2. RUNTIME VERIFICATION (Continuous)



LIMITATIONS



1. Kernel / Root Compromise

Escalation to root or kernel exploits breaks all guarantees.



2. TEE Tradeoffs

Hardware TEEs offer stronger isolation but are less deployable. Software TEEs are weaker against privileged attackers.

A5

3. A5 Deferred

Execution-state tampering is identified but not addressed. Advanced techniques (e.g., ZK proofs) are not yet practical at LLM scale.



4. No Experimental Validation

Detection rates, latency, and false positive rates have not been measured.

FUTURE WORK



1. Behavioral Check Implementation

Build and integrate the runtime behavioral attestation module.



2. Empirical Evaluation

Measure detection latency, accuracy, and false positive rates.



3. VM-Based Isolation

Explore VM and hypervisor-based isolation as a stronger software option.



4. Hardware TEE Integration

Leverage SGX, SEV-SNP, TrustZone and other TEEs where available.



5. A5 Integrity Coverage

Investigate practical techniques for execution-state tampering detection.

References and Acknowledgements

KEY PAPERS

- [1] M. Ammar, T. Caulfield, E. de Oliveira Nunes, et al., “SoK: Integrity, Attestation, and Auditing of Program Execution,” in Proc. IEEE Symp. Security and Privacy (S&P), 2025.
- [2] M. Schneider, R. J. Masti, S. Shinde, S. Čapkun, and R. Perez, “SoK: Hardware-supported Trusted Execution Environments,” arXiv preprint arXiv:2205.12742, 2022.
- [3] Y. Li, Z. Yang, P.-Y. Chen, Y. Yan, and Y. Zhang, “SoK: Understanding Design Choices and Pitfalls of Trusted Execution Environments,” in Proc. ACM ASIA Conf. Computer and Communications Security (ASIA CCS), 2024.
- [4] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and Implementation of a TCG-based Integrity Measurement Architecture,” in Proc. USENIX Security Symp., 2004.
- [5] R. Zhang, Y. Zhao, N. Javidnia, M. Zheng, and F. Koushanfar, “AttestLLM: Efficient Attestation Framework for Billion-scale On-device LLMs,” arXiv preprint arXiv:2509.06326, 2025.

ACKNOWLEDGEMENTS

Advisor: Dr. Eugene Vasserman Department of Computer Science, Kansas State University
Tooling: Diagrams in this presentation were generated with assistance from Claude (Anthropic) and ChatGPT (OpenAI).
Design template taken from Canva.com

THANK YOU!



QUESTIONS?